

Projet Algorithmie 3

La distance de Jaccard

De-backer Cuvelier Sébastien Llobregat Nicolas

Version du 09 Avril 2025

Table des matières

1	Organisation du travail	3
1.1	Outils utilisés	3
1.2	Répartition du travail	3
2	Structure du projet	3
2.1	Les modules	3
2.2	Gestion des options	4
2.3	Gestions des fichiers	5
2.4	Gestion de l'entrée standart	6
3	Choix d'implémentation	7
3.1	Stockage des mots avec holdall	7
3.1.1	Implémentation par LDSC	7
3.1.2	Implémentation par tableaux dynamiques	7
4	Calcul de la distance de Jaccard	7
5	Les difficultés rencontrées	8
5.1	Tentative d'utilisation de getopt	8
5.2	La fonction fscanf	9
6	Performances	9
6.1	Complexité en espace	9
6.2	Complexité en temps	10
7	Améliorations possibles	10

1 Organisation du travail

1.1 Outils utilisés

Git & Github: Pour la gestion du projet et des différentes versions de celui-ci nous avons fait le choix d'utiliser un logiciel de versionning: **GIT**. Ainsi nous avons pu travailler en simultané sur les différentes composantes du projet. La methodology était la suivante: Créations des tâches via les issues **Github**. Puis création d'une branche spécifique pour chaque tâche à réaliser. Une fois le travail réaliser nous ouvrons une **pull request** pour que notre binôme puisse analyser le travail de l'autre avant de mutualiser sur la branche principale.

Tests unitaires: Nous avons pour ambition d'utiliser les tests unitaires pour s'assurer que chaque fonction réalisait bien ce qu'elle devait, pour se faire nous avons commencer à utiliser une bibliothèque qui permettait assez simplement de les réalilser: **greatest**. Cependant il s'est avéré très chronophage et nous avons pris du retard sur ces tests ce qui leur a fait perdre toute utilité. Nous avons donc choisi de ne pas les inclure dans notre version finale du projet.

1.2 Répartition du travail

Mesure de la charge de travail: Au début du projet nous n'avions pas réellement conscience de la charge de travail à effectuer. Il aurait peut-être été intéressant de lors de la première séance de réaliser un planning avec un tableau de répartition des tâches pour définir une charge de travail équitable. Cependant le projet n'étant pas de grande envergure et étant en effectif réduis à deux personnes nous avons pu communiquer efficacement pour communiquer sur les tâches que nous étions entrain de réaliser.

2 Structure du projet

Origine: Lors de la conception de notre binôme nous avons décider de faire une lecture commune du sujet afin de savoir si nous avions la même vision. Ainsi nous avons organiser la structure du projet à l'origine puis nous nous sommes mis d'accord sur les modules à utiliser. Entre les deux implémentations suggérées soit l'utilisation de **tables de hachages** ou bien **d'abres binaires de recherches équilibrés**, c'est la deuxième qui nous a paru la plus adéquates. En effet il est plus facile de représenter les arbres et nous avons beaucoup plus travailler sur cette notion en cours.

2.1 Les modules

Les modules utilisés sont:

- Le module **avl**
- Le module **holdall**
- Le module **jdis**
- Le module **op**

Le module holdall: Nous avons eu recours au module **holdall** pour le stockage des mots, afin de séparer le traitement des mots de la gestion en mémoire. Nous aurions pu choisir d'utiliser la fonction **bst_dft_infix_apply_context** du module **avl** et par exemple libérer les ressources en même temps que le dernier parcours de chaque arbre. Nous avons égelment décider de tester cette solution en parallèle et nous en faisons référence dans la section 6 liée aux performances de l'implémentation proposée.

Le module jdis: Nous avons décider de créer un module spécifique pour gérer la liaison entre le module **avl** et notre utilisation, il consiste principalement à convertir un fichier ou l'entrée standart (voir 2.4) en un **avl**. En le calcul du cardinal de l'intersection des ensembles de mots stockés dans les arbres. Et bien évidemment au calcul de la distance de Jaccard entre chaque couple deux à deux distincts des ensembles représentés par les arbres sur laquelle nous reviendrons dans la sous section 4.

Le module op: Il permet principalement d'intervenir lors de l'affichage sur la sortie standart et permet ainsi de désengorger le main et le rendre plus clair. Il comprend donc la fonction qui affiche l'aide ainsi que le graph d'appartenance des mots.

2.2 Gestion des options

Pour ce qui est de la gestion des options, nous avons décidé d'implanter à la fois les options longues et les options courtes. Les paragraphes qui suivent préciseront ce que ces options font et comment elles le font.

Les options -? et -help: Ces options sont très simple à comprendre. Elles permettent d'afficher le menu d'aide pour l'utilisation de notre projet et des précisions sur le reste des options.

Les options -g et -graph: Ces options permettent d'afficher un graphe d'appartenance des mots dans les différents fichiers passés sur la ligne de commande. Cette fonction est divisé en deux partie.

```
static int scptr_display(context *ctx, const char *ref);
```

Cette fonction permet d'afficher une ligne du graph selon le schéma suivant:

Mot $\rightarrow$ Appartenance dans le premier fichier $\rightarrow \dots \rightarrow$ Appartenance dans le $n - i$ eme fichier

Cette partie là quand à elle affiche une ligne par mot présent dans l'union de tous les fichiers:

```
int graph_belonging(bst **t, bst *uni, int nb_file);
```

Les options -iVALUE et -initial=VALUE: Ces options permettent de fixer la taille des mots à **VALUE**. Celle-ci est traité au moment de la lecture des mots. Il est précisé plus tard dans ce compte rendu que nous avons dû créer notre propre version du fscanf pour pouvoir délimité la taille des mots lus sur le fichier.

Les options -p et -punctuation-like-space Ces options permet de ne pas compter les symboles correspondants à ceux des classes `isspace` et `ispunct`. C'est-à-dire que les mots composés avec des traits d'union sont maintenant considérés comme deux mots différents. Cette option aussi est traitée au moment de la lecture des mots en utilisant notre version de la fonction `fscanf`

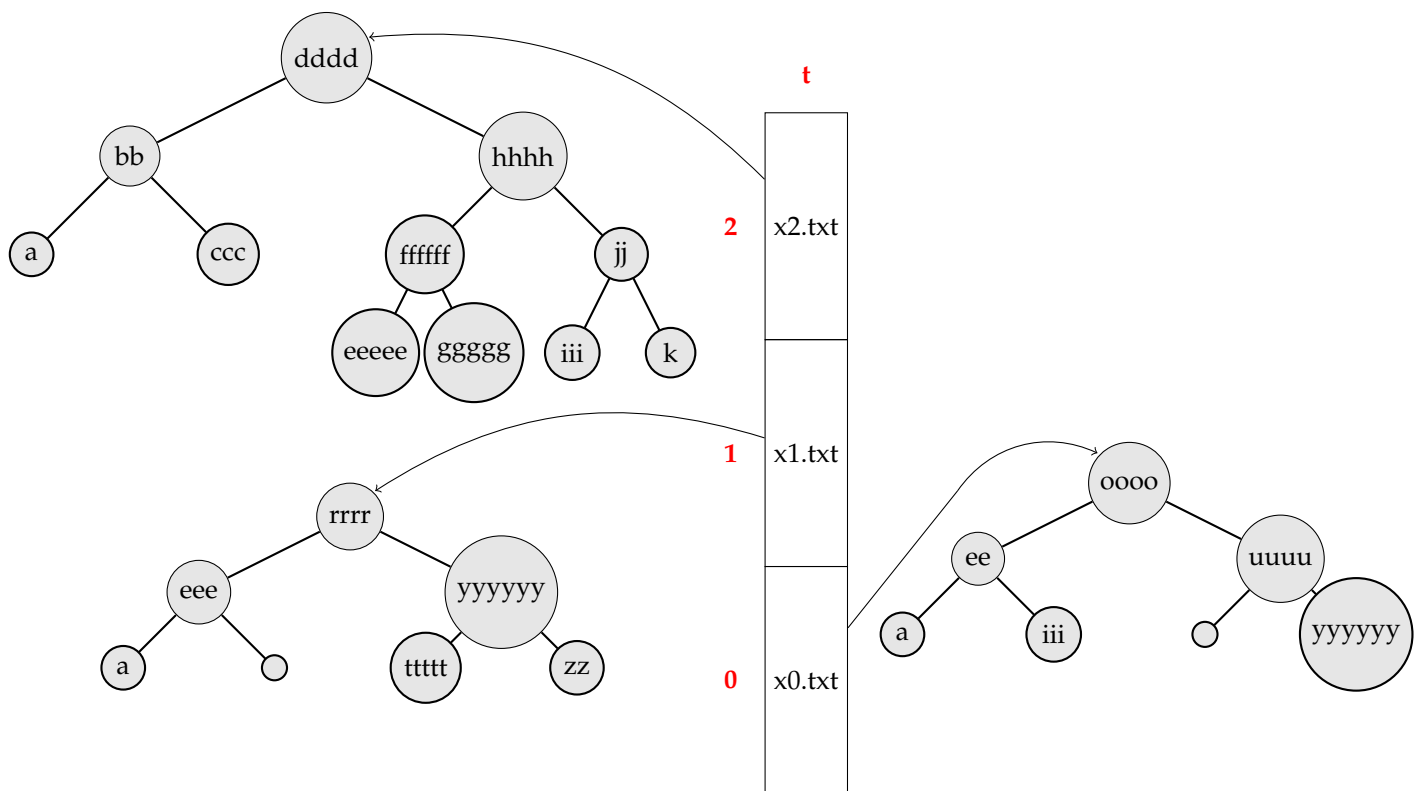
L'option -- Cette option permet d'échapper l'argument qui suit sur la ligne de commande. Si nous avons un fichier appelé `--help`, par exemple, cette option nous permet de traiter ce fichier comme un fichier et non comme l'option `-help` ci-dessus.

L'option - Cette dernière option permet de rentrer des mots sur l'entrée standard pour qu'ils soient traités comme un fichier. Nous parlerons un peu plus tard de la gestion de l'entrée standard.

2.3 Gestion des fichiers

Pour gérer les fichiers pris en arguments par le programme nous ouvrons simplement le fichier associé en mode lecture seule puis nous lisons (précisions 5.2) chaque mot et nous tentons de l'ajouter à l'arbre binaire associé. À la fin de la lecture nous avons un arbre contenant des pointeurs vers les mots. Il est à noter que pour un même fichier, on stocke en un et un seul exemplaire les mots apparaissant plusieurs fois dans le même fichier. Nous avons un tableau avec l'ensemble des arbres créés.

Exemple: Ainsi si nous exécutons la commande suivante: `./jdis x0.txt x1.txt x2.txt` où les fichiers `x0.txt` et `x1.txt` sont ceux donnés dans le sujet du projet, nous aurons la représentation suivante:



Exemple 1: Conversion d'un fichier en arbre

2.4 Gestion de l'entrée standard

Questionnement: Nous avons réfléchi comment traiter l'argument particulier '-', deux Choix s'offraient alors à nous, créer l'arbre depuis l'entrée standard directement et donc différencier très clairement les fichiers de l'entrée standard ou bien de stocker l'entrée standard dans un fichier puis effectuer le même traitement que sur les autres arguments sans distinction aucune. Dans un soucis d'homogénéité c'est cette deuxième implémentation qui a été sélectionnée.

Utilisation d'un fichier temporaire: La situation se prête parfaitement à un fichier temporaire, comme vu en cours d'Algorithmie 1, page 39 du support de cours et d'exercices de 2023:

La fonction `tmpfile`:

Crée un fichier temporaire ouvert en mode `w+b`. Ce fichier sera automatiquement supprimé lors de sa fermeture ou d'une terminaison normale du programme; En cas de succès, renvoie un pointeur vers le contrôleur du flot ; En cas d'échec, renvoie `NULL`.

d'où la création de la fonction statique **`stdin...to_file`** dans le module `jdis`. À la suite de ce traitement l'entrée standard est gérée exactement comme les fichiers normaux.

3 Choix d'implémentation

3.1 Stockage des mots avec holdall

Généralités: Deux versions de ce module ont été réfléchies pendant la création de ce projet. La première en utilisant le principe des listes dynamiques simplement chaînées et la seconde en utilisant les tableaux dynamiques.

3.1.1 Implémentation par LDSC

Au fil des cours d'algo nous avons appris les avantages et les inconvénients des LDSC. Ce qui nous intéressait particulièrement en premier lieu c'est la rapidité d'exécution. En effet puisque nous faisons des ajouts systématiques en tête de liste ces derniers se faisaient en temps constant. C'était rapide certes mais on utilise beaucoup d'espace dès que les fichiers deviennent imposants.

3.1.2 Implémentation par tableaux dynamiques

Pour résoudre le problème d'espace nous avons pensé aux tableaux dynamiques. En effet l'avantage, ici, est que les ajouts se font également en temps constant grâce à un accès direct aux cellules du tableau et nous n'avons pas besoin d'allouer toute une cellule de liste pour un seul mot; nous avons juste à allouer un tableau assez grand pour stocker les mots. Pour les petits fichiers cette implémentation s'est montrée très efficace mais pour les gros fichiers le programme passait énormément de temps à recopier et agrandir le tableau.

4 Calcul de la distance de Jaccard

Les paires distinctes: Nous avons maintenant notre tableau avec des pointeurs vers nos arbres, on peut rappeler que la taille d'un arbre s'effectue en temps constant dans le contrôleur. Nous allons avoir à calculer la distance de Jaccard entre chaque couple deux à deux distincts d'arbres. Pour cela il suffit d'avoir deux boucles imbriquées:

```
1 for (int i = 0; i < k - 1; ++i) {  
2     for (int j = i + 1; j < k; ++j) {  
3         . . .  
4     }  
5 }
```

La première boucle: nous commençons à $i = 0$ et allons jusqu'à $i = k - 2$ (car $i < k - 1$)

La deuxième boucle: pour chaque valeur de i nous commençons à $j = i + 1$ et va jusqu'à $j = k - 1$ (car $j < k$). L'idée ici est de parcourir chaque combinaison unique de i et j où i est strictement inférieur à j . Cela permet de créer une paire unique sans répéter les mêmes éléments dans les deux positions.

Rappel sur la distance de Jaccard: Nous devons maintenant effectuer le calcul de la distance de Jaccard, petit rappel sur la formule: La **distance de Jaccard** entre deux ensembles A et B est définie par la formule suivante:

$$J_{\delta}(A, B) = \begin{cases} 0 & \text{si } A = \emptyset \wedge B = \emptyset, \\ 1 - \frac{|A \cap B|}{|A \cup B|} & \text{sinon.} \end{cases}$$

Calcul du cardinal de l'union Nous avons donc besoin du cardinal de l'intersection ainsi que le cardinal de l'union de chaque couple, pour cela nous allons en réalité ne calculer que le cardinal de l'intersection. En effet nous allons utiliser une petite astuce de la théorie des ensembles qui nous dit que: Le cardinal de l'union de deux ensembles n'est rien d'autre que la somme de leurs cardinaux respectifs moins le cardinal de l'intersection. On peut facilement s'en rendre compte ici:

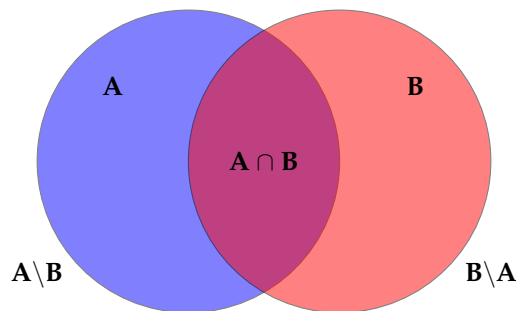


Schéma 1: Généralités sur les ensembles

Calcul du cardinal de l'intersection Nous devons alors calculer l'intersection des deux ensembles, pour cela nous allons de nouveau utiliser la fonction `bst.dft.infix.apply.context` du module `avl`. Cette fois-ci nous allons utiliser un contexte qui sera une structure contenant un compteur ainsi que le plus petit des deux ensembles. Ainsi nous allons parcourir le plus grand des deux et rechercher dans le plus petit la référence grâce à la fonction `bst.search` du module `avl`. Si on la trouve alors on ajoute un au compteur cela veut dire qu'elle se trouve dans les deux sinon non. A la fin dans le contexte nous aurons le cardinal de l'ensemble.

Calcul final: Il nous reste plus qu'à rassembler les éléments, ainsi nous avons: Un moins le cardinal de l'intersection divisé par la taille des deux arbres moins le cardinal de l'intersection ce qui nous donne la distance de Jaccard entre les deux ensembles de mots de la paire de fichiers. On peut alors réitérer sur toutes les paires.

5 Les difficultés rencontrées

5.1 Tentative d'utilisation de getopt

La bibliothèque getopt: En premier lieu nous nous sommes intéressés à la bibliothèque `getopt` qui nous permettait de parser la ligne de commande pour trouver les options notamment grâce à `optind` et `optarg`. Nous pouvions également préciser quelles options nécessitaient des paramètres comme pour l'option qui permet de fixer la taille maximale des mots `-iVALUE` ou `-initial=VALUE`. Grâce à `getopt` nous avons pu implanter les options très rapidement, mais nous avons rencontré un problème lorsque nous voulions implanter l'option `'-'` permettant d'échapper

le nom d'un fichier. En effet, lorsque nous utilisons cette option nous avons remarqué quelle échappait tous les paramètres qui suivaient. Nous avons d'abord cherché une solution en remplaçant **getopt** par **getopt_long**. Ce qui nous compliquait la tâche. Finalement il a été choisi de parser nous même la ligne de commande pour avoir un contrôle total sur son traitement.

5.2 La fonction `fscanf`

utilisation de `fscanf`: Afin de créer les arbres comme nous l'avons décrit dans la section 2.3 nous avons besoin de lire dans les fichiers, pour se faire il convient d'usage d'utiliser la fonction **`fscanf`**, et c'est le choix que nous avons fait lors de la première partie du projet cependant, après avoir essayé d'implémenter l'option -p (resp. -punctuation-like-space) nous avons un problème de compréhension de la fonction. Il fallait de plus reparser le buffer que nous prenions afin de l'analyser par la suite. En outre cela revenait à lire chacun des caractères deux fois et donc de doubler la complexité temporelle. Il a été alors naturellement convenu de coder notre propre fonction de lecture d'où l'apparition de la fonction statique **`hm_fscanf`** (hm pour home maid). Ainsi nous lisons caractère par caractère dans le fichier donné par la tête de lecture en paramètre grâce à la fonction **`fgetc`**. Elle permet de gérer en une seule fois le cas où nous devons prendre en compte la ponctuation ou non.

6 Performances

Analyse de quelques données: Nous allons analyser quelques données afin d'avoir une idée de la performance de notre exécutable, nous verrons ensuite une étude théorique afin de conformer nos observations, enfin nous donnerons quelques pistes d'améliorations.

6.1 Complexité en espace

Fichiers traités	Holdall (LDSC)	Holdall (TAB DYN)	Nb de mots	Taille des fichiers
Les misérables / Domjuan	676000	611000	64497	3.1Mo / 100ko
fr / abeeillmrsss / tartuffe	1.7M	1.2M	401574	3.9Mo / 3.1Mo / 116ko
toto0 / toto1 / toto2 / toto3	463	343	127	159o / 192o / 320o / 167o

6.2 Complexité en temps

Fichiers traités	Holdall (LDSC)	Holdall (Tableau dynamiques)	Nombre de mots	Taille des fichiers
Les misérables / Domjuan	0.9s	2.1s	64497	3.1Mo / 100ko
fr / abeillmrsss / tartuffe	1.5s	45s	401574	3.9Mo / 3.1Mo / 116ko
toto0 / toto1 / toto2 / toto3	0.004s	0.2s	127	159o / 192o / 320o / 167o

7 Améliorations possibles